MODERATED LEARNING MANAGEMENT SYSTEM

Release v1.1

Student: Molapo Setsumi

Student No.: 202322042

Repository: github.com/molaposetsomi5/SC4433-MLMS

1. Abstract

This individual project developed the Moderated Learning Management System (MLMS), an offline, menu-driven C++17 console application with a local SQLite database for a secondary school. Four roles are supported: Administrator, Lecturer, Head of Department (HoD) and Student. The defining rule is that no student sees a mark until the HoD of that department has moderated and released it. The system covers user and course administration, enrolment, assignments, submissions and resubmissions, individual and class-wide extensions, marking with feedback, moderation, release and an append-only audit log of every mark change.

The engineering goal was a maintainable, secure and testable system with traceable requirements, not only a working program. The implementation uses C++17, CMake, SQLite, libsodium (Argon2id), GoogleTest, cppcheck, clang-tidy and GitHub Actions. The measured outcome is 128 of 128 automated tests passing, 95.9% line coverage of the business logic, zero static-analysis findings and an 80 ms login with 800 students stored.

2. Development process and project management (Phase 1)

The project used the Waterfall model with V-Model verification, because the brief fixes the phases and gate reviews and the requirements could be agreed early. A **feasibility study** found the project technically feasible (free, mature tools; SQLite needs no server), operationally feasible (it mirrors the existing mark-moderate-release workflow), economically feasible (no licence or hardware cost), legally feasible if privacy is a Must requirement (Lesotho Data Protection Act 2011), and feasible on schedule with strict scope control. The recommendation was to proceed, with a slippage rule: drop Could, then Should requirements before any Must requirement.

The **work breakdown structure** has 9 phases and 39 work packages totalling 105 hours over 7 weeks. The **Gantt chart** shows finish-to-start dependencies, milestones (plan, Gates 1-3, releases v1.0 and v1.1) and the critical path from feasibility through requirements, design, implementation, system testing and release to the final report. The **risk register** scored 14 risks (likelihood x impact); the highest were stakeholder conflicts and schedule overrun (16) and security flaws and a failed Gate 2 (15). It was re-scored at every gate: by Gate 3 the requirement, design, performance and coverage risks were closed, and the slippage rule was used once to drop the Could requirement FR-20.

Figure 1. Gantt chart with dependencies, milestones and critical path. View diagram ↗

Full documents: Phase 1 plan (WBS, schedule, risks) ↗ | Updated risk register ↗

3. Requirements engineering (Phase 2)

Six stakeholders were analysed: the four system roles plus the Principal and IT officer, whose policies decide acceptance. **Fifteen conflicts and ambiguities (C1-C15)** were resolved and recorded. The key ones: marks stay hidden until HoD release, but the HoD releases a whole assignment in one step so release is quick (C1); work is marked only after the deadline so students can still resubmit (C2); a late day is any started 24-hour period and the penalty is 10% of the maximum mark per day with a floor of zero (C3); late work is always accepted and gets zero only after three days (C4); an extension moves that student's effective deadline and needs a reason (C5); unlimited resubmission before the deadline with all versions kept (C6); the HoD may approve, amend with a reason, or return (C7); and "fast" became 2 seconds with 800 students (C11).

The **SRS** follows ISO/IEC/IEEE 29148 and contains 20 functional requirements (FR-01 to FR-20) and 10 measurable non-functional requirements (NFR-01 to NFR-10), each with a unique ID, MoSCoW priority and acceptance criterion; for example NFR-01 requires login and the assignment list within 2 seconds with 800 students stored. The **use case model** covers all four actors with a generalised User actor, <<include>> for audit recording,

late penalty and moderation-before-release, and <<extend>> for resubmission. Three use cases are fully dressed with preconditions, main, alternative and exception flows, postconditions and business rules: UC-01 Submit assignment, UC-02 Mark submission and provide feedback, and UC-03 Moderate and release marks.

Figure 2. Use case diagram: four actors with include, extend and generalisation. View diagram ↗

Full document: Phase 2 stakeholder analysis, SRS and use cases ↗

4. Software architecture and detailed design (Phase 3)

A **layered architecture** separates presentation (role menus), application services, domain rules, data access and SQLite storage. Each layer calls only the one below; the console never runs SQL, and domain rules have no console or database dependency, so they can be unit-tested alone. AppContext is the composition root that builds every object once. **Model-View-Controller was rejected**: MVC pays off with complex or multiple views, whereas this system has one simple console view per role, so it would add controllers without benefit. A client-server design was rejected because the IT officer requires offline, single-computer operation.

Figure 3. Layered architecture and component diagram. View diagram ↗

4.1 Class design, design patterns and SOLID

An abstract **User** class has four subclasses (Administrator, Lecturer, HeadOfDepartment, Student) that override hasPermission(), so role checks use polymorphism. Four patterns are applied: **Strategy** (LatePenaltyPolicy / StandardLatePenaltyPolicy) because the penalty rule is the policy most likely to change; **Repository** (I*Repository interfaces with SQLite implementations) so services never contain SQL and can be tested with an in-memory database; **Factory** (UserFactory) to turn a stored role into the right subclass in one place; and **Dependency Injection** through constructors so tests can supply a fixed clock and fast hasher. SOLID is followed: one responsibility per service, new penalty rules without changing services (Open/Closed), substitutable User subclasses, small per-entity repository interfaces, and services depending on abstractions rather than SQLite.

Figure 4. Class diagram: domain model, User hierarchy, Strategy and Factory. View diagram ↗

Figure 5. Class diagram: services, repositories and dependency injection. View diagram ↗

4.2 Behavioural models

The **state machine** separates submission status (SUBMITTED or LATE) from mark status (AWAITING_MODERATION, RETURNED, APPROVED, RELEASED); a mark becomes RELEASED only from APPROVED. The **swimlane activity diagram** follows Student, System, Lecturer and HoD from submission to released mark, including the return-for-correction loop and refused release. The **sequence diagrams** trace Submit assignment (first submission, resubmission before the deadline, rejected late resubmission, rollback on failure) and Moderate and release marks (department check, decision saved with its audit record, all-or-nothing release).

Figure 6. Submission and mark state machine. View diagram ↗

Figure 7. Activity diagram with swimlanes. View diagram ↗

Figure 8. Sequence: Submit assignment (late and resubmission paths). View diagram ↗

Figure 9. Sequence: Moderate and release marks. View diagram ↗

4.3 Database design

The database has 13 tables in **Third Normal Form**: departments, users, courses, lecturer_courses and enrolments (junction tables with composite keys), assignments, submissions, submission_versions, extensions, class_extensions (v1.1), marks, moderation_decisions and audit_logs. Primary keys, foreign keys, UNIQUE and CHECK constraints prevent orphans, duplicate submissions and invalid statuses or marks, and two triggers block UPDATE and DELETE on audit_logs. Classes map one-to-one to tables, except the four User subclasses, which share a users table with a role column (UserFactory rebuilds the subclass), and the many-to-many links, which need tables but no classes. The data dictionary and SQL schema are in the Phase 3 document and src/data/Schema.cpp.

Figure 10. Entity relationship diagram (schema version 2). View diagram ↗

Full documents: Phase 3 design document ↗ | All diagrams with PlantUML sources ↗

5. Implementation, security and persistence (Phase 4)

The code (about 3,800 lines in 55 files) follows the layers: src/ui, src/services, src/domain, src/data, src/security and src/app. All code except main() is built as a library so the tests run exactly the production code. Every service first calls PermissionPolicy, which checks the role and the resource scope (lecturer teaches the course, HoD owns the department, student is enrolled and sees only their own records), so hiding a menu item is never the only protection. Diagrams were redrawn from the final code; the only differences from the Phase 3 design are a merged PermissionPolicy, an added DeadlineResolver (shared effective-deadline logic) and the CR1 class extension.

5.1 Business rules and access control

- **Late penalty:** lateDays() counts started 24-hour periods; 10% of the maximum mark per day for days 1-3, zero after day 3, never below zero (40/50 two days late gives 30).
- **Resubmission** only before the effective deadline and while unmarked; every version is kept. **Marking** only after the student's effective deadline.
- **Moderation:** approve, amend with reason (penalty recalculated) or return with reason, only within the HoD's department. **Release** is all-or-nothing per assignment and closes it to new submissions.
- **Extensions:** one individual extension per student plus one class extension; the later deadline wins.

5.2 Security and persistence

Passwords are stored only as **salted Argon2id hashes** via libsodium's crypto_pwhash_str, chosen because it is a well-reviewed library with a one-call API and Argon2id is memory-hard against guessing attacks. Accounts lock for 15 minutes after five failures, and a wrong username and wrong password give the same message. SQLite is reached only through repositories with **parameterised queries**; RAII wrappers always release handles; foreign keys are enabled; and every mark change commits together with its **audit record** (actor, timestamp, old value, new value, reason) inside one transaction. Exceptions derived from MlmsError are caught by the console, so invalid input or a database error never crashes the program. The code follows a documented coding standard.

Evidence: Source and build guide ↗ | Coding standard ↗ | Demo transcript ↗

6. Requirements-to-design traceability

- Authentication and roles (FR-01/02) → AuthService, PermissionPolicy, User hierarchy → AuthTest, UserPermissions, ST-08
- Administration and enrolment (FR-03-05) → User/CourseManagementService → AdminTest, ST-09
- Submission, extension and late penalty (FR-07-10) → SubmissionService, DeadlineResolver, LatePenaltyPolicy → SubmissionTest, 19 boundary tests, ST-01/02
- Marking, moderation and release (FR-11-13) → Marking, Moderation and ReleaseService → MarkingTest, ModerationTest, ST-04-07
- Audit trail (FR-14/18, NFR-06) → AuditService, audit_logs triggers → AuditTest
- CR1 class extension (FR-21) → ExtensionService.grantClass, class_extensions → ClassExtensionTest (12)

Evidence: Full traceability matrix (Appendix) ↗

7. Verification, validation and quality assurance (Phases 5-6)

The **test plan** maps each V-Model level to its artefact with entry and exit criteria: unit tests verify the detailed design, integration tests the architecture (services, repositories and an in-memory SQLite database with a controllable clock), system tests the requirements through the use cases including exception flows, and acceptance the stakeholder needs. **Equivalence partitioning** split submission time into on time, late within three days and beyond three days; **boundary values** tested 1 second before, exactly at, and 1 second after the deadline, 24 h, 24 h + 1 s, 72 h and 72 h + 1 s, and marks at -0.5, 0, the maximum and just above.

GoogleTest runs **128 tests, all passing**, in CI and on Windows: unit, integration and 12 scripted system tests that drive the real console (for example late submission to released mark, HoD return, wrong-department denial). **Line coverage of the business-logic layer is 95.9%** (902/941 lines, gcovr), above the 70% target. The 800-student performance test measured an 80 ms login and 0.5 ms assignment list (NFR-01). The **defect log** records 10 defects

with severity, cause, fix and retest; none was critical, and all are closed. Coverage shows executed lines, not exhaustive correctness, and acceptance was a role-play walkthrough rather than a trial with real staff.

For quality assurance, **cppcheck** reported 23 findings and **clang-tidy** 13; all were fixed or suppressed with a written reason, leaving zero. **Lizard** metrics: 287 functions, average cyclomatic complexity 2.5; parseDateTime was split from 16 to 4, and only a flat admin menu switch (16) remains above 15. A **self code review** against a 15-point checklist found, among others, a test with no assertions, which was rewritten.

Evidence: Test plan ↗ | Test results ↗ | 128 passing (screenshot) ↗ | Defect log ↗ | QA report ↗ | Code review ↗

8. Configuration management and CI

Git was used with **feature branches** (feature/domain-and-data, feature/services, feature/console-ui, feature/tests, feature/ci-and-qa, feature/cr1-class-extension) merged into main with no-fast-forward merges, commit messages of the form "area: what changed" with the requirement or CR number, and **tagged releases v1.0 and v1.1**. A **GitHub Actions** pipeline runs on every push, pull request and tag: build, all tests, performance check, cppcheck and clang-tidy (fail on any finding), coverage (fail below 70%) and metrics. It proved its value after the first v1.1 push, when a lost suppression comment failed the cppcheck step; the fix was pushed and CI went green (defect D-10).

Figure 11. Configuration management and CI pipeline. View diagram ↗

Evidence: CI runs ↗ | Workflow file ↗ | CM plan ↗

9. Deployment (Phase 7)

The deployment diagram shows one school computer (Windows or Ubuntu) running the mlms executable with SQLite and libsodium, a local mlms.db file and a weekly backup on a USB drive; no network is needed. The **installation guide** lets a technician build, test and set up the system in about 30 minutes and covers first-run administrator setup, backup, restore and upgrading. The **user manual** has a section for each role, and the **release notes** list features and known limitations (single computer only, password visible while typed, FR-20 not built). Release v1.0 is tagged and published; the demo accounts are for evaluation only.

Figure 12. UML deployment diagram. View diagram ↗

Evidence: Installation guide ↗ | User manuals ↗ | Release notes ↗ | Release v1.0 ↗

10. Software maintenance: change request CR1 (Phase 8)

CR1 asked that a lecturer can extend the deadline for a whole class and that extensions appear on the student's assignment list. It was classified as **perfective maintenance**: v1.0 had no fault (not corrective) and the environment had not changed (not adaptive); users requested a new capability. **Impact analysis** identified a new FR-21 and changes to FR-07 and FR-09; the use case, class, ERD and submit-sequence diagrams; the ClassExtension, DeadlineResolver, ExtensionService, AssignmentService and ConsoleUI classes; a new class_extensions table (schema version 1 to 2 with automatic upgrade); and every deadline-related test.

On feature/cr1-class-extension, a class extension is one row per assignment, so later-enrolled students also receive it, and the effective deadline is the latest of the original, class and individual deadlines, so a class extension never shortens a personal one. Students see "extended for the whole class (was ...)". Twelve new tests were added and **all 128 tests passed (the 116 existing tests unchanged)**; a real v1.0 database file was upgraded with no data lost. The change was merged and released as **v1.1**.

Evidence: CR1 impact analysis and regression results ↗ | Release v1.1 ↗

11. Professional, legal and ethical considerations

The system holds personal data of minors, so the **Lesotho Data Protection Act 2011** applies: data must be limited to its purpose, kept accurate and protected. The design applies least privilege (the administrator cannot see marks; students see only their own released results), department-scoped audit access, hashed passwords and an offline local database; the school must still secure the computer and backups. **Assessment records** are protected because every change is attributed and append-only, and transactions prevent a mark being saved without its audit record.

On **accessibility**, the console suits screen readers and keyboard-only users and uses plain messages, but offers no text-size or contrast settings; a future GUI should follow WCAG 2.1. On **fairness**, one tested Strategy class applies the same penalty to everyone, late work still receives feedback, and extensions need a reason the HoD can see; the system relies on a correct computer clock and does not exclude weekends. Tool licences were checked and limitations are reported honestly.

12. Critical evaluation and conclusion

The project delivered every Must requirement with a design, code and tests that match and are traceable. Its strengths are the careful conflict resolution in Phase 2, which turned straight into code and tests; separation of concerns that kept CR1 small; moderation and audit enforced in the service layer and the database; and evidence-based quality (128/128 tests, 95.9% coverage, zero analyser findings, green CI). Limitations are single-computer storage, text or path-based submissions, visible password typing and the dropped FR-20. Future work would add hidden password input, managed file storage, an accessible GUI and a client-server version for a school network.

References and repository evidence

Sommerville, I. (2016) Software engineering. 10th edn. Pearson. | Gamma, E. et al. (1994) Design patterns. Addison-Wesley. | ISO/IEC/IEEE 29148:2018 Requirements engineering. ISO. | Kingdom of Lesotho (2012) Data Protection Act, 2011 (Act No. 5 of 2012). | Biryukov, A., Dinu, D. and Khovratovich, D. (2016) 'Argon2', IEEE EuroS&P, pp. 292-302. | libsodium, SQLite, GoogleTest and CMake documentation (accessed 1 November 2026). | W3C (2018) WCAG 2.1.

Appendices: Traceability matrix ↗ | Full test results ↗ | User manuals ↗ | Updated risk register ↗

Source and releases: GitHub repository ↗ | v1.0 ↗ | v1.1 ↗ | Demonstration video ↗